fractal

Session 2:

Build Your First PoC (Part-1)

3 Usable Problem Statements

ExpenseFlow

What it does:

- Submit an expense, store it, normalize to INR
- Generate an AI spending insight via Claude API
- Approver accepts or rejects the submission

DischargeFlow

What it does:

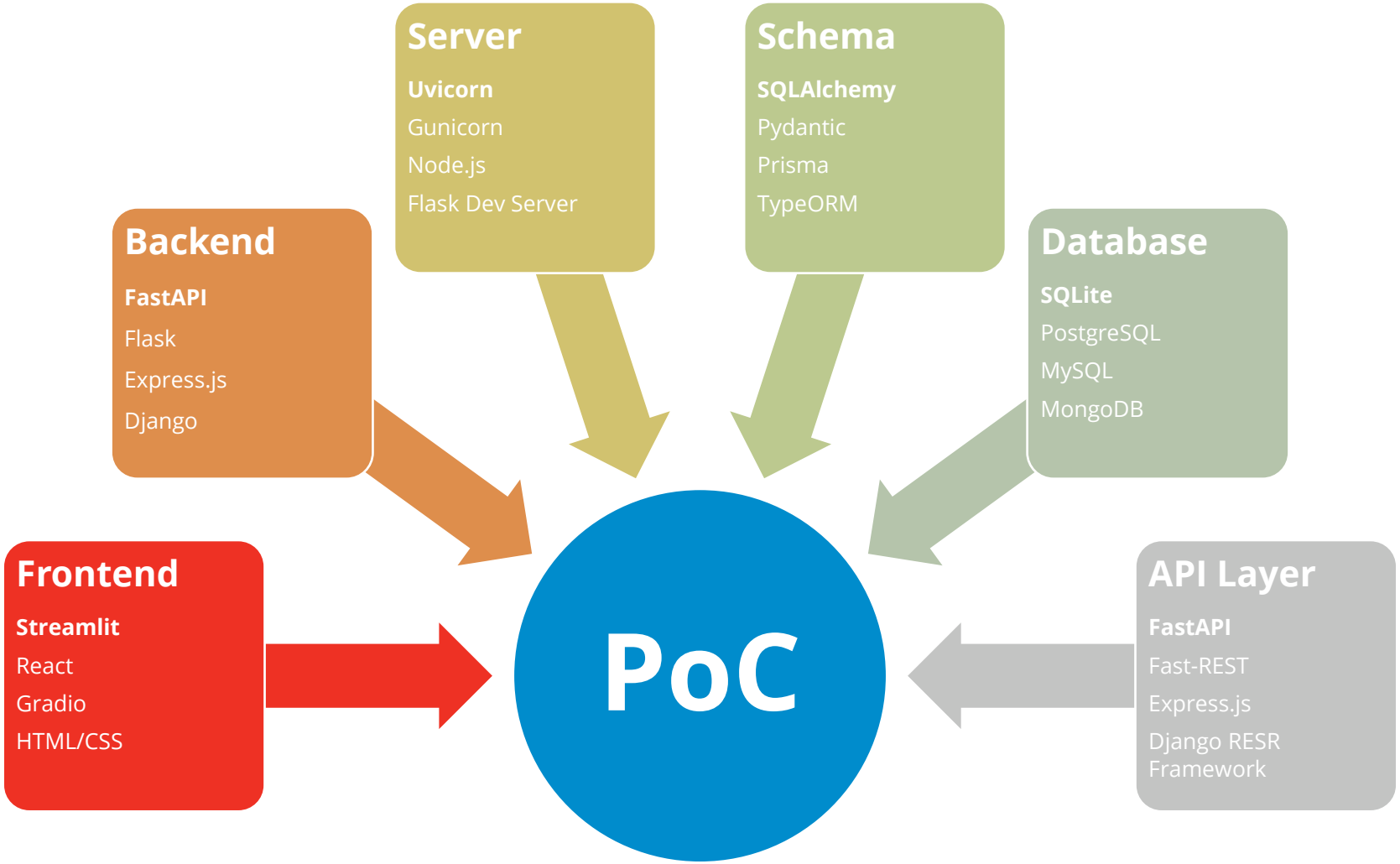
- Clinician submits a structured discharge record
- Claude API drafts a plain-language patient summary
- Reviewing clinician approves or rejects the draft

ReturnFlow

What it does:

- Customer raises a return against an order line
- Return window eligibility is computed server-side
- Claude API generates a returns insight. Agent approves or rejects with a disposition

Components of a PoC



01 SCOPING AND BRIEFING YOUR PoC

Questions You Ask Before Building a PoC

The PoC you build should answer something you or your team actually needs to know

> **Business Outcome**
> *What business problem am I trying to solve?*

> **User Understanding**
> *Who is the primary user of this solution?*

> **Scope and Focus**
> *What is the smallest version of this idea that can demonstrate value?*

> **Data and Inputs**
> *What information/data does the solution need?*

Ask learners to write some similar questions they think before starting on a PoC.

From Brief to CLAUDE.md

Your brief becomes the standing instruction that keeps Claude Code focused throughout the build.

PoC BRIEF

```
What: Expense submission API that stores
      expenses, normalises amounts
      to INR, and generates an AI
      spending insight per submission

Who: Finance professional submitting and
      approving expenses — CLI workflow

Success: Expense stored, amount in INR minor
        units, one insight generated,
        CLI output shows result

Out of scope: No UI, no email, no Postgres,
             no multi-currency UI
```

CLAUDE.md

```
## Project Overview
ExpenseFlow PoC — expense submission,
normalisation to INR minor units,
and Claude-generated spending insight.
Used by finance staff — CLI only.

## Tech Stack
Python 3.11 · FastAPI · SQLite
anthropic SDK · Entry: main.py

## Constraints
Money as integer minor units (paise).
All amounts normalised to INR on write.
CLI output only. No UI.

## Do Not Touch
Do not add a UI, email, or Postgres.
Do not change the money convention.
```


fractal

Write Your PoC Brief and CLAUDE.md

HANDS-ON LAB

You already chose your problem statement. Now make it buildable.

1. **Open your PoC brief template**

   Use the four-element format: What · Who · Success · Out of scope

2. **Write your brief for [PROBLEM STATEMENT]**

   One sentence per element. Pass the three scope questions before moving on.

3. **Run /init to generate a starter CLAUDE.md**

   Then edit it using your brief as the source of truth.

4. **Verify Claude has read it**

   Ask Claude to name your project purpose and one constraint — without reading any file.

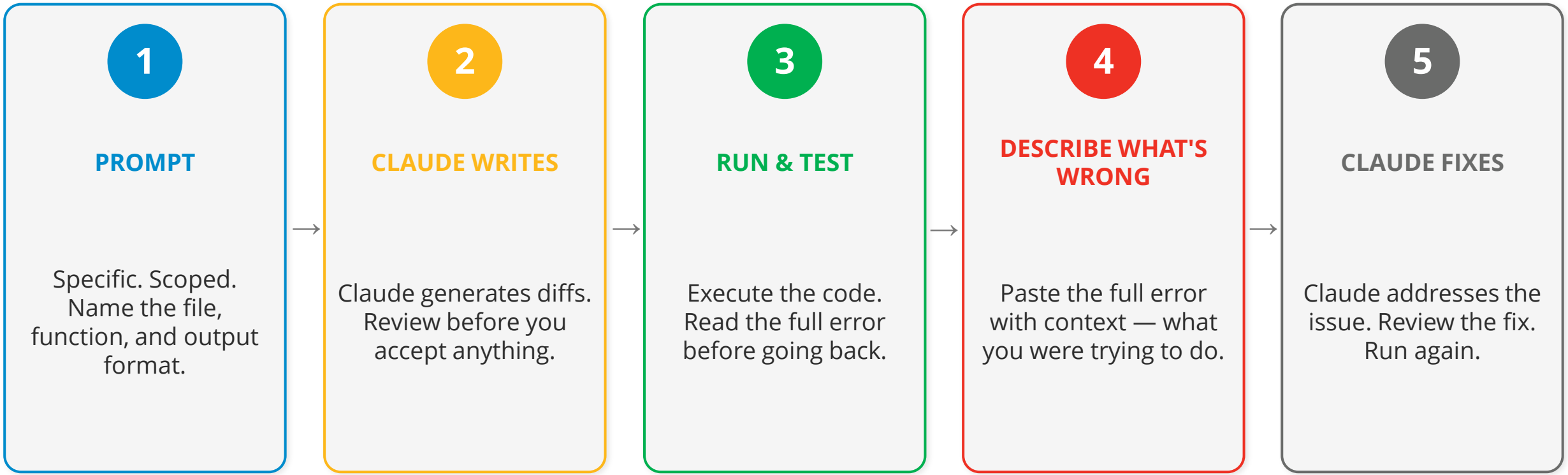
✓ **SUCCESS: Brief written · Scope test passed · CLAUDE.md created · Claude correctly names your project and constraints**

fractal

02 THE BUILD LOOP

The Build Loop — Five Steps

Each Claude response is one step — not a final answer. The loop is the method.



Repeat *until the code passes your success criterion — not until it 'feels done'.*

Step 1: Prompt — Working With APIs

Most PoCs connect to an API. Claude Code scaffolds the entire call — connection, authentication, and response — without you memorising documentation.

Tell Claude the API name, what you want back, and how errors should be handled. It writes the connection code. You direct the shape — Claude handles the boilerplate.

01 Scaffold the call

Name the API, the endpoint, and what you expect back. Claude writes the connection code.

"Connect to the CRM API and return all accounts with a last_contact field"

02 Handle authentication

Ask Claude to use an environment variable for the API key. Never hardcode credentials.

"Use the API key from the environment variable CRM_API_KEY"

03 Parse the response

Tell Claude which fields you need. It extracts and formats them without you reading the schema.

"Return only: account name, owner, and days since last contact"

Step 2: Claude Writes — Review Before You Accept

Claude does not save anything until you say yes. Your job at this step is to read what it proposes — not to approve it automatically.

A diff is Claude's proposal — a side-by-side view showing exactly what it wants to add, change, or delete. Green means added. Red means removed. Nothing changes on your machine until you press Y to accept.

Did Claude do what I asked?

✓ The new function matches your prompt — correct file, correct output format

✗ Claude added extra logic or features you did not ask for

Reject the extra parts. One thing at a time.

Does anything look out of place?

✓ The change is small and contained — only the lines you expected

✗ Claude touched files or sections you did not mention in your prompt

Ask Claude to explain why it touched those lines before accepting.

Can I explain what changed?

✓ You can describe in one sentence what the diff does and why

✗ You are not sure what changed or why — but it looks fine

If you cannot explain it, do not accept it yet.

fractal

Step 3: Run & Test — Reading the Result

You accepted the diff. Now run it. The output tells you what to do next — whether it worked, broke, or produced the wrong result.

Running the code is not optional. Reading the output and deciding what to report back to Claude is the skill that separates fast builds from slow ones.

✓

It worked

The output matches your success criterion. Check it carefully before moving on.

Read the output. If it is correct, move to the next feature. Do not add more before confirming this one works.

✗

It broke with an error

An error message appeared. This is the most useful outcome — it tells you exactly where to look.

Copy the complete error message. Do not trim it. Go to Step 4.

?

It ran but the result is wrong

No error, but the output does not match what you expected. This is the trickiest case.

Compare actual output to expected output. Write down the difference in plain English. Go to Step 4.

fractal

Step 4: Describe What's Wrong — Handle the Error Right

When something breaks, paste the full error into Claude Code with context about what you were trying to do — not just the error message.

Something will break. That is not a problem — it is the process. What you paste back to Claude determines whether the fix works first time or after five more rounds.

✗ What people paste

✗ Pasting only the error message — no context

✗ Asking Claude to 'fix it' without saying what 'it' is

✗ Accepting the first fix without re-running the code

✗ Starting a new conversation and losing all context

Claude has no idea where to look or what to fix.

→

✓ What actually works

✓ Paste the full error — never trim it

✓ Add one sentence: what you were trying to do

✓ Let Claude name the cause before fixing it

✓ Run the fix — if it fails again, repeat the loop

Claude now knows exactly where to look and what to fix.

fractal

Step 5: Claude Fixes — This Is Not the End

Claude proposes a fix. Your job is the same as Step 2: review before you accept. Then run it again to confirm the fix actually worked.

A fix that makes the error message disappear is not necessarily a fix. It is a hypothesis. Your job is to verify it before moving on.

01

Review the fix first

Before accepting, read what Claude changed. Does the fix address the root cause, or does it just suppress the error message? Ask yourself: could this break something else?

02

Accept and run

Press Y to accept. Run the code immediately. Do not move on before running it. The fix is unverified until the code runs.

03

Check the symptom is gone

The original error or wrong output should no longer appear. If it does, you are back at Step 4. Describe what is still wrong and repeat the loop.

04

Check nothing else broke

Run the full output, not just the fixed part. A fix that solves one thing and silently breaks another is not a fix. Session 3 covers how to write a test that locks this in permanently.

Repeat the loop until the code passes your success criterion. Step 5 is not the end — it leads back to Step 3.

Thank You.

A strategic partner to the most admired Fortune 500® companies globally, we help power every human decision in the enterprise by bringing advanced analytics & AI, engineering and design.

www.fractal.ai